

Methodology: Automated Backend Generation from Triton IR to a Declarative Toy ISA

0. Grounding and Design Philosophy

This methodology is assembled from four source pipelines, each contributing a distinct piece:

- **TAIDL / ACT** [Jain et al.] contribute the *idea of a declarative, tensor-level ISA description* consumed by an automated generator, and the phase split between a **compute path** (instruction selection over tensor operators) and a **memory/addressing path** (buffer allocation). We borrow the concept, not the machinery — ACT's equality-saturation instruction selection and constraint-programming memory allocation are explicitly out of scope; we substitute greedy structural pattern matching.
- **TensorLift** [Gao et al.] contributes the *pipeline shape*: canonicalization → idiom detection → loop/shape reconstruction → structured metadata emission, plus its verification

philosophy (prove what's cheap to prove, differentially validate the rest, and annotate-don't-rewrite so failures are legible instead of silently wrong).

- **Triton-MTIA** [Zhu et al.] contributes the *target compiler shape*: Frontend (ttir, unmodified) → Middle-end (map ops to architecture, decide DMA-vs-scalar) → Backend (architecture-specific lowering) → Codegen, and its evaluation discipline of separating GEMM-class kernels from long-tail kernels and reporting coverage statistics, not just speed.
- **The SegFault brief** contributes the *scope cut*: fixed-shape kernel, toy declarative ISA, greedy pattern matching only, NumPy differential validation instead of SMT proof.

The project is the missing cell in the landscape table these four papers jointly establish: nothing today takes a declarative ISA description and automatically emits a Triton-IR-consuming backend. Everything below is designed so that cell gets filled with a working, if narrow, artifact — not a partial attempt at ACT's full generality.

1. Research & Landscape Grounding

Before any code, fix precisely what is being reused vs. reinvented, per source:

1.1. Re-derive TensorLift's Table 2 (eight-pass pipeline) as a checklist, and mark which passes have a direct analogue in this project and which don't:

- Canonicalization (A1/A2) → *does* apply: Triton IR has its own bit-manip-adjacent noise (redundant `tt.splat` / `tt.broadcast` chains, repeated `tt.addptr` arithmetic) that should be canonicalized before pattern matching, same motivation as TensorLift's sign-extension collapsing.
- Idiom detection (B3–B5: MAC, control specialization, clamp) → *partial* analogue: MAC/dot detection is directly relevant (§6); clamp/saturation detection is deferred (toy ISA has no fixed-point saturation initially).
- Loop/shape reconstruction (C6/C7) → **inverted**, and this inversion is worth stating explicitly in writeups: TensorLift *builds* `scf.for` reductions out of flattened bit-level arithmetic because RTL extraction destroys structure. Triton's compiler already emits `scf.for` with `iter_args` around `tt.dot` — structure is present by construction. The methodology step is *recognition*, not *reconstruction*. This is the single biggest reason the 5-day budget is plausible: an entire TensorLift

phase is replaced by a much cheaper structural match.

- Metadata emission (D8) → *does* apply: classify each memref-like argument (`tt.load` / `tt.store` targets) by role (operand A, operand B, output, DMA source/dest) before assembly, exactly as TensorLift classifies scratchpad vs. accumulator roles.

1.2. **Re-derive ACT's problem formalization** (§3 of the ACT paper: tiled kernel as input, computational vs. addressing attributes, soundness/completeness definitions) and explicitly *narrow* it for this project:

- Adopt the α (computational) / β (addressing) attribute split conceptually: the MAC op's tile shape and dtype are computational attributes fixed by the toy ISA; the memory strides/offsets are addressing attributes resolved per-kernel.
- Explicitly do **not** adopt ACT's partially-instantiated-instruction (pii) machinery or its equality-saturation search — document this as a scoped-out generalization axis, not a missed requirement.
- Adopt ACT's soundness/completeness vocabulary for the completeness report in §10: every kernel op is either lowered or explicitly flagged opaque; nothing is silently miscompiled.

1.3. **Re-derive the Triton-MTIA four-stage compiler flow** (Fig. 4 of that paper) as the target architecture

skeleton for the generator itself:

- Frontend = ttir extraction (§4).
- Middle-end = idiom/pattern detection and op-to-toy-ISA-instruction mapping (§6), directly analogous to MTIA's "map ttir onto hardware engines" stage and its DMA-vs-scalar-load decision, which we replicate as DMA-vs-unsupported.
- Backend = toy ISA assembly with resolved addressing (§7).
- Codegen = emission of the final instruction stream + the NumPy-executable form used for validation (§9).

1.4. Catalogue triton-shared's documented failure modes (fragile pointer/mask/offset analysis, crashes on unstructured access, monolithic hard-to-extend pass — cited directly by its own maintainers) as a **negative reference**: every design decision in §5–§6 should be checked against "does this reproduce a known triton-shared failure mode at small scale?" If a chosen kernel's pointer arithmetic can't be proven structured, the methodology's answer is to route it to the opaque fallback (§8), not to attempt general pointer analysis. This is a direct, deliberate scope boundary inherited from a documented real-world failure.

1.5. Output of this stage: a one-page internal "inheritance map" document (which design element came from which paper, and which capability was deliberately left out and why). This becomes the backbone of any future writeup and prevents scope creep mid-build, since every proposed addition can be checked against it.

2. Toy ISA Description Language Design

Design a minimal, TAIDL-flavored declarative schema – not TAIDL itself, but structurally recognizable as its miniature.

2.1. **Data model declarations**, mirroring TAIDL's `add_data_model` calls: a flat memory space (no scratchpad/accumulator distinction initially, since bank/tiling constraints are exactly the complexity TensorLift's RTL extraction exists to recover, and this project starts downstream of that problem).

2.2. **Instruction declarations**, mirroring TAIDL's `add_instruction` + `add_semantics` pattern, expressed as YAML rather than a Python DSL to keep the assembler simple:

- One DMA instruction (load/store, block-structured: base + shape + stride, matching ACT's

addressing-attribute concept).

- One MAC/dot instruction (fixed tile shape, matching ACT's computational-attribute concept), with an explicit accumulate flag so multi-iteration reduction loops map to repeated invocations rather than requiring the ISA to encode a loop primitive itself.
- One elementwise instruction (add/relu) for the epilogue-fusion generalization step (§6.4).

2.3. **Explicit non-goals in the schema**, written down so they don't get silently assumed later: no saturation/clamp semantics, no multi-bank memory, no control-flow instructions beyond the fixed reduction pattern. Each non-goal is paired with the TensorLift or ACT feature it corresponds to, so the schema doc reads as "here is what we are not yet doing, and here is who does it."

2.4. Validate the schema itself by hand-writing the toy-ISA program for the reference matmul kernel *before* the generator exists. This hand-written program becomes the ground-truth comparison target for §9 (benchmarking) and §10 (testing) — directly mirroring how TensorLift and ACT both benchmark generated output against a hand-written reference rather than only against a simulator.

3. ttir Extraction Methodology

3.1. **Environment pinning:** fix the Triton package version explicitly and record it; ttir textual syntax is not guaranteed stable across releases, and reproducibility of the parser (§5) depends on a fixed grammar.

3.2. **Compilation methodology without live**

accelerator hardware: extract ttir via

`triton.compile()` with an explicitly constructed target object rather than relying on an active driver/GPU. This must be validated once, up front, as its own checkpoint — if it fails, the entire methodology's GPU-free premise fails with it, so this is treated as a go/no-go gate before any generator code is written, not a background assumption.

3.3. **Kernel corpus design**, deliberately graduated in complexity so the pipeline is tested incrementally rather than validated only on the hardest case at the end — mirroring how TensorLift evaluates separately on compute-dominated modules (PE) vs. control-heavy modules (Load/Store Controllers) and reports different reduction numbers for each:

- Tier 0 (control/structure only): vector add — pure structured DMA in/out, no compute idiom, used to validate the memory-access recognition pass in isolation.

- Tier 1 (core target): fixed-shape single-block `matmul ( tt.dot` inside `scf.for` with `iter_args )` – the primary demonstration kernel.
- Tier 2 (epilogue generalization): `matmul` with a fused elementwise op (bias-add or ReLU) – validates that idiom detection composes rather than being a single hard-coded template.
- Tier 3 (negative control): a kernel with an intentionally unstructured access pattern (e.g., modulo-based pointer wraparound, the exact nonportable pattern the MTIA paper's §8.1 identifies as needing rewriting) – this kernel is *expected* to hit the opaque fallback, and confirming that it does so cleanly (no crash, no silent wrong output) is itself a required, documented test outcome, not a failure.

3.4. **Fixture snapshotting:** persist the extracted `ttir` text for every corpus kernel to disk immediately after successful extraction. All downstream work (parsing, pattern matching, testing) runs against these frozen fixtures rather than re-invoking `triton.compile()` each time – this decouples generator development from Triton/version flakiness and gives a stable regression baseline for §10.

4. IR Parsing & Normalization

4.1. Build a minimal ttir-text-to-graph parser: tokenize SSA value definitions, operand lists, and result types line by line (the ttir dialect surface needed here — `tt.*`, `arith.*`, `scf.for` / `scf.yield` — is small enough that a hand-rolled parser is both feasible and preferable to depending on MLIR Python bindings, which would reintroduce the exact build-from-source burden the methodology is designed to avoid per §3.1–3.2).

4.2. Construct an internal def-use graph: each SSA value is a node; operands are edges. This is the direct analogue of TensorLift's decision to preserve structure (its choice to keep `scf.if` / named memrefs rather than lower to LLVM branches/phi nodes) — the parser must **not** discard the structural information (loc names, `iter_args` correspondence, `scf.for` bounds) that §6's pattern matching depends on.

4.3. **Canonicalization pass** (direct analogue of TensorLift Phase A): collapse redundant `tt.splat` → `tt.broadcast` chains and fold repeated `tt.addptr` accumulation into a single symbolic stride expression per pointer. This is the step that turns the verbose, mechanically-generated pointer arithmetic seen in real ttir dumps into a compact strided-access descriptor, exactly as TensorLift's Phase A turns sign-extension chains into single casts. Success criterion: line/node count reduction on the

canonicalized graph, reported the same way
TensorLift reports its 92.9%/41.2% figures (§9.2),
scaled appropriately to this project's much smaller IR.

5. Structured Memory Access Recognition

5.1. Define a **structured-access descriptor**: base pointer, per-dimension stride, per-dimension shape — the same three-field addressing-attribute shape ACT's memory allocation phase operates on and the same shape triton-shared's pointer analysis attempts to recover (and sometimes fails to, per its own maintainers' documented crashes).

5.2. Implement recognition as a bounded, syntax-driven walk (not general symbolic pointer analysis): trace each `tt.load / tt.store`'s address operand back through a fixed, small set of expected op shapes (`tt.addptr` $\leftarrow$ `tt.splat / tt.broadcast` $\leftarrow$ `arith.muli / arith.addi` over `tt.make_range / tt.get_program_id`). If the walk terminates in a recognized shape, emit a structured-access descriptor; if it doesn't within a small fixed number of hops, mark the access unstructured and route it to the opaque fallback (§8) rather than attempting to generalize the walk — this bound is the concrete mechanism that keeps the project inside 5

days and is also the exact discipline that avoids reproducing triton-shared's fragility.

5.3. This pass is tested independently, before compute-idiom detection is attempted, against Tier 0 and Tier 3 of the corpus (§3.3): Tier 0 must fully resolve; Tier 3 must cleanly fail to resolve. Both outcomes are asserted in an automated test, not eyeballed.

6. Compute Idiom Detection

6.1. **MAC/reduction idiom:** match an `scf.for` whose `iter_args` include an accumulator tensor, whose body contains exactly one `tt.dot` consuming that accumulator and two operands each already resolved to a structured-access descriptor (§5), and whose `scf.yield` re-threads the accumulator. This is a direct structural match against the pattern already empirically confirmed to be Triton's actual compiled output for a tiled matmul (`%acc:N = scf.for ... iter_args(...) { %acc' = tt.dot %a, %b, %acc ...; scf.yield ... }`) — the methodology exploits the fact that Triton's own compiler has already done the loop-structuring work TensorLift has to reconstruct from scratch (§1.1).

6.2. **Attribute extraction:** from the matched idiom, extract tile shape and dtype (the computational/a

attributes in ACT's vocabulary) and the two operands' structured-access descriptors (the addressing/ β attributes) — this split is what makes §7's assembly step a direct lookup rather than a search.

6.3. Elementwise epilogue idiom (Tier 2 generalization): match a fixed small set of post-loop elementwise ops (`arith.addf` against a broadcast bias, or a max-against-zero ReLU pattern) applied to the accumulator before the final `tt.store`. This is deliberately implemented as a *second, independent* pattern rule rather than folded into the MAC rule, so that adding future idioms is additive rather than requiring the MAC matcher to be rewritten — directly following TensorLift's own design principle that passes should annotate independently rather than intertwine (§3.2 of the TensorLift paper: "passes annotate rather than rewrite").

6.4. Every successfully matched idiom is tagged on the graph with a metadata attribute analogous to TensorLift's `tensorlift.mac / taidl.*` annotations (e.g., `toyisa.mac`, `toyisa.dma_role`), so that §7 (assembly) is a pure consumer of annotations and contains no pattern-matching logic itself — preserving the same annotate/assemble separation TensorReduce and TAIDL/ACT both rely on.

7. Backend Assembly (Toy ISA Codegen)

7.1. Split assembly into the same two paths ACT and TensorLift's Stage 3 both use: a **compute path** (annotated MAC/elementwise idioms → toy ISA instruction templates, filling in the tile-shape/dtype computational attributes fixed at schema-design time in §2.2) and a **DMA path** (annotated structured-access descriptors → toy ISA load/store instructions, filling in base/stride/shape addressing attributes).

7.2. Ordering constraints (load operands before the MAC that consumes them; MAC before the store that drains it) are recovered directly from the def-use graph's topological order — no separate FSM-inference step is needed here (unlike TensorLift §3.3, which must reconstruct instruction ordering from controller RTL because RTL exposes no explicit dataflow graph; Triton IR already *is* a dataflow graph, so this ordering is free).

7.3. Emit the final instruction stream as a simple linear list (opcode + resolved operands), plus a parallel human-readable disassembly for debugging and for the completeness report in §9.

8. Opaque Fallback Strategy

8.1. Any op node that reaches assembly without a `toyisa.*` annotation is emitted as an explicit `UNSUPPORTED(<op name>)` marker rather than being dropped, mistranslated, or causing a crash — this is the direct adoption of TensorLift's "an instruction that fails to match a template simply receives no annotation... Stage 3 falls back to opaque semantics rather than producing incorrect [output]" principle, and of ACT's soundness requirement that a generator must never silently miscompile.

8.2. The fallback path is a **first-class, tested outcome**, not an afterthought: Tier 3 of the kernel corpus (§3.3) exists specifically to exercise it, and the test suite (§10) asserts that unsupported ops are surfaced, not swallowed.

9. Benchmarking Methodology

Benchmarking here is deliberately modeled on the *shape* of TensorLift's and ACT's evaluations, scaled to what a 5-day toy project can actually measure honestly — no claim is made to reproduce their absolute numbers.

9.1. **Coverage metric** (analogue of TensorLift §4.4, "Specification Completeness"): for each corpus kernel, report the fraction of ttir op nodes that received a `toyisa.*` annotation vs. the fraction routed to the

opaque fallback. Report per-tier, not only aggregated, exactly as TensorLift reports per-module rather than only an overall number — this surfaces where the approach is strong (Tier 1/2, compute-dominated) vs. where it is intentionally weak (Tier 3, unstructured access), which is a more honest and more presentable result than a single blended percentage.

9.2. IR reduction metric (analogue of TensorLift §4.2/Table 3): report raw ttir node count vs. canonicalized graph node count (§4.3) vs. final toy ISA instruction count, per kernel. This is the same "before/after lines" story TensorLift tells with Figure 4, at a scale appropriate to this project's IR sizes.

9.3. Reference-match metric (analogue of ACT §8.3–§8.6 and TensorLift §4.4's comparison against the hand-written TAIDL artifact): compare the generator's output instruction sequence against the hand-written toy-ISA program authored in §2.4 for the same kernel — checking both structural equivalence (same instruction count and shape) and, via §11, numerical equivalence. Any discrepancy is itself a diagnostic: either the generator found a legitimately different (but valid) lowering, or a bug — both are worth recording.

9.4. Generation latency: wall-clock time from ttir input to toy ISA output, per kernel, reported the same way TensorLift reports its one-time per-accelerator pipeline cost (§4.2, "Pipeline cost") — framed explicitly

as a one-time, per-kernel-shape cost rather than a per-compilation runtime cost, since that is the correct framing this class of tool actually has.

9.5. Assemble all of the above into a single results table before writing any narrative summary — the table is the artifact; prose is generated from it, not the other way around, which keeps the eventual writeup honest about what was actually measured.

10. Testing Strategy

Testing is layered to match the pipeline's own stages, so a failure localizes to one component rather than surfacing only as "the end-to-end demo is wrong."

10.1. **Fixture-based regression tests** (§3.4): every corpus kernel's frozen ttir text is a checked-in fixture; parsing (§4), memory-access recognition (§5), and idiom detection (§6) each have unit tests running directly against these fixtures, independent of a live Triton install being available in CI.

10.2. **Per-pass unit tests:**

- Canonicalization: assert specific known-redundant patterns (splat → broadcast chains) collapse to the expected minimal form.

- Memory-access recognition: assert Tier 0/1/2 kernels resolve to the expected structured-access descriptors (exact base/stride/shape values, not just "resolved: yes/no"); assert Tier 3 resolves to "unstructured."
- Idiom detection: assert the MAC pattern is found exactly once in Tier 1/2 kernels with the expected operand bindings; assert it is *not* falsely triggered on Tier 0 (which has no `tt.dot` at all) — a true-negative test, which is as important as the true-positive ones and easy to skip under time pressure.

10.3. **End-to-end integration tests:** for each corpus tier, run the full `ttir` → toy-ISA pipeline and assert (a) the expected coverage outcome from §9.1, (b) that Tier 3 produces at least one `UNSUPPORTED` marker and zero crashes, and (c) that Tiers 1–2's generated instruction streams execute against the NumPy interpreter (§11) without runtime errors.

10.4. **Numerical differential testing** (methodology's substitute for TensorLift's Z3/golden-simulator dual-track, explicitly documented as such rather than left implicit): for Tiers 1–2, execute the generated toy ISA program on random input tensors via the NumPy interpreter and compare against `torch` eager execution of the equivalent operation, across a small matrix of shapes/dtypes chosen to stress the pattern matcher (at minimum: the exact block shape used in

extraction, one non-square shape, and one shape at a different block-size configuration) — mirroring TensorLift's own emphasis on validating against golden execution data wherever a formal proof isn't feasible, and mirroring ACT's practice of validating against real numerical execution rather than structural inspection alone.

10.5. Regression on canonicalization idempotency: assert that running the canonicalization pass twice produces identical output to running it once — a cheap, high-value check that catches an entire class of pass-ordering bugs before they propagate into idiom detection.

10.6. Negative/fuzz pass: construct at least one additional kernel not in the original corpus, written last (after the pipeline is otherwise working) specifically to try to break assumptions baked into §5–§6 (e.g., a matmul with a non-power-of-two block size, or an extra no-op elementwise identity op inserted mid-graph). This is not expected to fully pass — its purpose is to confirm the failure mode is the opaque fallback (§8) and not a crash or silent wrong output, which is the actual correctness bar this project is targeting.

11. Numerical Validation Harness

11.1. Implement the toy ISA as directly interpretable: DMA-load/store map to NumPy array slicing with the resolved stride/shape descriptor from §5–§7; MAC maps to `np.matmul` / `np.tensordot` over the annotated tile shape; elementwise maps to the corresponding NumPy ufunc.

11.2. This interpreter is deliberately kept separate from the "real" assembly output (§7.3) – it consumes the same instruction stream a real toy-accelerator simulator would, which keeps the validation honest (it is testing the generated program, not a shortcut around it).

11.3. Report parity as exact match (integer/low-precision paths) or within-tolerance match (float paths), and record the tolerance used – this small detail matters because silently choosing a loose tolerance would undermine the correctness claim, and TensorLift/ACT are both explicit about their own verification bounds for exactly this reason.

12. Documentation of Scope Boundaries

12.1. Maintain, alongside the code, a running "known limitations" list generated directly from every explicit non-goal noted in §1–§8 (no saturation, no multi-bank

memory, bounded-hop pointer walk only, two idiom classes only, greedy matching with no search/backtracking). This list is not an apology — it is the deliverable that lets the project be presented honestly as *a demonstration of the missing automated-generator layer* rather than as a competitor to ACT's generality, which is the correct and defensible framing given the actual scope covered.

12.2. Cross-reference every limitation against which of the four source papers solves it at full generality (e.g., "arbitrary pointer arithmetic: triton-shared attempts this and documents its own failures; full generality would require ACT-style equality saturation over addressing expressions") — this turns the limitations list into evidence that the scope cuts were deliberate engineering decisions grounded in the literature, not gaps discovered too late to fix.